

An Explicit Infinite Model Refuting Ulrich’s **u4** as a Single Axiom for Positive Implication

Byungwoong Yoo
Independent Researcher

Strengthened verification candidate – not for release until all gates pass
Lean 4.33, leanchecker, nanoda, axiom-audit, and symbolic-check suite
20 August 2026

Abstract

Fitelson and Peltier (2026) eliminated three of Ulrich’s four remaining 15-symbol candidates for a single axiom of positive implicational logic, leaving

$$((x \rightarrow y) \rightarrow z) \rightarrow ((y \rightarrow (z \rightarrow u)) \rightarrow (y \rightarrow u))$$

as an open case. We construct an explicit infinite model satisfying every substitution instance of this formula and closed under modus ponens, while falsifying reflexivity $q \rightarrow q$. The model is the union of the substitution instances of a recursively defined family of formula schemes. The decisive countermodel property, uniform-substitution closure, the induced proof system, and an internal separation from the standard positive-implication basis are formalized in Lean 4.33.0. The strengthened release is gated on the Lean kernel, **leanchecker**, the independent Rust checker **nanoda**, a compiled-environment axiom audit, and a separate Python symbolic checker. No release claim is made for a commit unless every recorded gate succeeds.

1 Background

Carew Meredith found single axioms of length 17 for positive implication. Dolph Ulrich reduced the possibility of a shorter axiom to four formulas of length 15. Fitelson and Peltier used saturation-based theorem proving to eliminate the first three and reported that the fourth candidate,

$$\Phi = ((x \rightarrow y) \rightarrow z) \rightarrow ((y \rightarrow (z \rightarrow u)) \rightarrow (y \rightarrow u)), \quad (1)$$

remained open. They further established that the shortest single axioms have length 17 if and only if Φ is not a single axiom.

The proof below is semantic. Its domain is the free algebra of finite implicational formulas. The theorem predicate is a particular substitution- closed set that is also closed under modus ponens.

2 Recursive schemes

Write

$$C(A, B, R) := (A \rightarrow (B \rightarrow R)) \rightarrow (A \rightarrow R),$$

so that

$$\Phi = ((x \rightarrow y) \rightarrow z) \rightarrow C(y, z, u).$$

Choose pairwise distinct variables

$$a, b, c, r_2, r_3, \dots$$

and define

$$T_1 = \Phi,$$

$$A_2 = a, \quad B_2 = C(b, a, c), \quad T_2 = C(A_2, B_2, r_2).$$

For $n \geq 2$, recursively set

$$A_{n+1} = B_n \rightarrow r_n, \quad B_{n+1} = A_n \rightarrow r_n,$$

$$T_{n+1} = C(A_{n+1}, B_{n+1}, r_{n+1}).$$

For $n \geq 2$, put

$$D_n = A_n \rightarrow (B_n \rightarrow r_n),$$

so that

$$T_n = D_n \rightarrow (A_n \rightarrow r_n).$$

For a formula scheme S , let $\text{Inst}(S)$ denote the set of all finite substitution instances of S .

3 The interlocking property

For finite formulas P, Q , let $E(P, Q)$ mean that no substitution σ and finite formula W satisfy

$$\sigma(P) = \sigma(Q) \rightarrow W.$$

Lemma 1 (Interlocking). *For every $n \geq 2$,*

$$E(A_n, B_n) \quad \text{and} \quad E(B_n, A_n).$$

Proof. At $n = 2$,

$$A_2 = a, \quad B_2 = ((b \rightarrow (a \rightarrow c)) \rightarrow (b \rightarrow c)).$$

If $\sigma(a) = \sigma(B_2) \rightarrow W$, then the finite formula $\sigma(a)$ occurs as a proper subformula of its own right-hand side. This is impossible. Conversely, if $\sigma(B_2) = \sigma(a) \rightarrow W$, injectivity of implication forces

$$\sigma(b \rightarrow (a \rightarrow c)) = \sigma(a),$$

which again makes $\sigma(a)$ a proper subformula of itself.

Assume both assertions at n . Since

$$A_{n+1} = B_n \rightarrow r_n, \quad B_{n+1} = A_n \rightarrow r_n,$$

an equality

$$\sigma(A_{n+1}) = \sigma(B_{n+1}) \rightarrow W$$

would imply

$$\sigma(B_n) = \sigma(A_n) \rightarrow \sigma(r_n),$$

contradicting $E(B_n, A_n)$. The converse direction is symmetric. □

4 The major-premise obstruction

Lemma 2. *For every $m \geq 2$ and $n \geq 1$, the schemes D_m and T_n have no common substitution instance.*

Proof. First let $n \geq 2$, and suppose

$$\sigma(D_m) = \tau(T_n).$$

Expanding both sides and using injectivity of implication gives

$$\sigma(A_m) = \tau(A_n) \rightarrow (\tau(B_n) \rightarrow \tau(r_n))$$

and

$$\sigma(B_m) = \tau(A_n).$$

Thus

$$\sigma(A_m) = \sigma(B_m) \rightarrow (\tau(B_n) \rightarrow \tau(r_n)),$$

contrary to Lemma 1.

Now let $n = 1$. If $m \geq 3$, then

$$A_m = B_{m-1} \rightarrow r_{m-1}, \quad B_m = A_{m-1} \rightarrow r_{m-1}.$$

Matching D_m with T_1 forces both

$$\sigma(r_{m-1}) = \tau(z) \quad \text{and} \quad \sigma(r_{m-1}) = \tau(z \rightarrow u),$$

hence $\tau(z) = \tau(z) \rightarrow \tau(u)$, impossible for a finite formula.

For $m = 2$, a common instance of

$$D_2 = a \rightarrow (C(b, a, c) \rightarrow r_2)$$

and T_1 would imply

$$\sigma(a) = \tau((x \rightarrow y) \rightarrow z)$$

and

$$\sigma(C(b, a, c)) = \tau(y \rightarrow (z \rightarrow u)).$$

The second equality yields

$$\tau(y) = \sigma(b \rightarrow (a \rightarrow c)), \quad \tau(z) = \sigma(b), \quad \tau(u) = \sigma(c).$$

Substitution into the first equality makes $\sigma(a)$ contain itself as a proper subformula, a contradiction. $\square$

5 The infinite model

Define

$$\mathcal{P} = \bigcup_{n \geq 1} \text{Inst}(T_n).$$

Interpret the unary theorem predicate p on the free algebra of finite implicational formulas by

$$p(F) \iff F \in \mathcal{P}.$$

Every instance of $\Phi = T_1$ belongs to $\mathcal{P}$. It remains to prove closure under modus ponens.

Lemma 3. *If $X \in \mathcal{P}$ and $X \rightarrow Y \in \mathcal{P}$, then $Y \in \mathcal{P}$.*

Proof. Choose $n, m \geq 1$ and substitutions α, β with

$$X = \alpha(T_n), \quad X \rightarrow Y = \beta(T_m).$$

If $m \geq 2$, the antecedent of $\beta(T_m)$ is $\beta(D_m)$. Then X would be a common instance of D_m and T_n , contrary to Lemma 2. Hence $m = 1$, so

$$X = \beta((x \rightarrow y) \rightarrow z), \quad Y = \beta(C(y, z, u)).$$

If $n = 1$, rename the variables of the minor copy of T_1 as x', y', z', u' . Comparison of

$$\alpha(T_1) = \beta((x \rightarrow y) \rightarrow z)$$

gives

$$\beta(y) = \alpha(z'), \quad \beta(z) = \alpha(C(y', z', u')).$$

Therefore

$$Y = C(\alpha(z'), \alpha(C(y', z', u')), \beta(u)),$$

an instance of $T_2 = C(a, C(b, a, c), r_2)$.

If $n \geq 2$, comparison of

$$\alpha(T_n) = \beta((x \rightarrow y) \rightarrow z)$$

gives

$$\beta(y) = \alpha(B_n \rightarrow r_n), \quad \beta(z) = \alpha(A_n \rightarrow r_n).$$

Hence

$$Y = C(\alpha(B_n \rightarrow r_n), \alpha(A_n \rightarrow r_n), \beta(u)),$$

which is an instance of T_{n+1} . Thus $Y \in \mathcal{P}$. □

6 Failure of reflexivity

Lemma 4. *No formula $Q \rightarrow Q$ belongs to $\mathcal{P}$.*

Proof. For $n \geq 2$, an instance of

$$T_n = (A_n \rightarrow (B_n \rightarrow r_n)) \rightarrow (A_n \rightarrow r_n)$$

could equal $Q \rightarrow Q$ only if

$$\sigma(B_n \rightarrow r_n) = \sigma(r_n).$$

This would make the finite formula $\sigma(r_n)$ contain itself as a proper subformula.

For $n = 1$, equality of the antecedent and consequent of an instance of T_1 forces

$$\sigma(y) = \sigma(z \rightarrow u), \quad \sigma(z) = \sigma(y \rightarrow u),$$

a contradiction by strict size increase in both directions. □

Theorem 5. *Ulrich's formula (1) is not a single axiom for positive implicational logic under substitution and modus ponens.*

Proof. The interpretation above satisfies every substitution instance of Φ , and Lemma 3 proves that it satisfies modus ponens. Lemma 4 shows that it falsifies reflexivity $Q \rightarrow Q$, which is a theorem of positive implication. Therefore reflexivity is not derivable from Φ , so Φ is not a single axiom. □

Corollary 6. *Combining Theorem 5 with the elimination and classification results established by Fitelson and Peltier, the minimum length of a single axiom for positive implication is 17 symbols.*

7 Formal certificate and release gate

The evidence bundle includes `lean/U4Formal.lean`. It defines the free algebra of finite implicational formulas, the exact formula Φ , and an explicit predicate P . The Lean kernel-checked theorem is

```
theorem U4Formal.u4_countermodel_exists :  
  Exists fun pred : Fml -> Prop =>  
    (forall x y z u, pred (Phi x y z u)) /\  
    (forall X Y, pred X -> pred (I X Y) -> pred Y) /\  
    (forall q, Not (pred (I q q)))
```

The strengthened file additionally formalizes arbitrary uniform substitution, proves that every syntactic `u4` derivation is sound in the model, and derives reflexivity from the standard K and S schemes. Its final theorem exhibits a positive-implication theorem that is not derivable from `u4`:

```
theorem U4Formal.u4_not_axiomatizes_positive_implication :  
  Exists fun f : Fml => PositiveDerivable f /\ Not (U4Derivable f)
```

The source scan forbids `sorry`, `admit`, `custom axiom`, and `unsafe`; the compiled-environment audit is the decisive check against hidden trust escapes. The CI artifact records the exact Git commit, all checker outcomes, direct compiler output, source files, and SHA-256 hashes. This manuscript must be distributed only with the all-green artifact for the identical commit.

8 Executable cross-checks

The release workflow performs the following mutually reinforcing checks.

Check	Range	Result
Lean build and direct compilation	Lean 4.33.0	required
Lean environment recheck	<code>leanchecker</code>	required
Independent Rust type check	<code>nanoda</code> , no <code>sorryAx</code>	required
Compiled axiom audit	standard allowlist only	required
Python recurrence/interlocking/major obstruction	$T_1, \dots, T_{100}$	required
Python exhaustive binary-tree enumeration	through 12 leaves	required

At 12 leaves there are $C_{11} = 58,786$ binary application-tree shapes. Exactly one was typable: the right comb. These checks support, but do not replace, the all- n proof above.

9 Verification status and claim boundary

The official 2026 source reported `u4` as open. Exact-formula and keyword searches located no later indexed resolution. This does not rule out unpublished or non-indexed prior work.

The earlier public deposit records a Lean 4.30.0 kernel check of the core countermodel. The strengthened release adds explicit substitution and derivability, an internal K/S separation theorem, and multiple independent compiled checks. The mathematical exposition, the connection to the prior reduction, external specialist review, journal peer review, and worldwide priority remain separate questions. After every strengthened gate succeeds, the defensible description is:

Lean-checked explicit infinite countermodel, with independent environment and type-checker audits, showing that Ulrich’s `u4` is not a single axiom; external specialist review and comparative priority audit remain pending.

AI-use disclosure. GPT-5.6 Pro substantially assisted with proof exploration, adversarial checking, independent program drafting, literature-search assistance, and manuscript preparation. The human author, Byungwoong Yoo, finalized the mathematical argument and the submitted package, reviewed the claim and evidence, and assumes full responsibility for the accuracy and final wording. Verification and replay details are documented separately in the supplied logs and checkers.

References

- [1] B. Fitelson and N. Peltier, “Applying Saturation-Based Theorem Proving to Open Problems in Positive Implicational Logic,” *Journal of Automated Reasoning*, vol. 70, article 5, 2026. doi:10.1007/s10817-026-09752-1.
- [2] J. R. Hindley and D. Meredith, “Principal Type-Schemes and Condensed Detachment,” *Journal of Symbolic Logic*, vol. 55, no. 1, pp. 90–105, 1990. doi:10.2307/2274956.
- [3] J. A. Kalman, “Condensed Detachment as a Rule of Inference,” *Studia Logica*, vol. 42, no. 4, pp. 443–451, 1983. doi:10.1007/BF01371632.
- [4] D. Ulrich, “New Single Axioms for Positive Implication,” *Bulletin of the Section of Logic*, vol. 28, pp. 39–42, 1999.
- [5] D. Ulrich, “A Legacy Recalled and a Tradition Continued,” *Journal of Automated Reasoning*, vol. 27, no. 2, pp. 97–122, 2001.
- [6] C. A. Meredith, “A Single Axiom of Positive Logic,” *Journal of Computing Systems*, vol. 1, no. 3, pp. 169–170, 1953.